PRODUCT REQUIREMENT DOCUMENT (PRD)

Offline AI Tutor for Government Exam Preparation

Project Name:	Offline AI Tutor (MVP)	**Target Field:**	SSC + Banking Exams
Author:	Aman, Founder	**Date:**	June 15, 2026
Company:	Solvor Pvt. Ltd.	**Document Version:**	V1.0.0-STABLE

---

1. Product Vision & Goals

The **Offline AI Tutor** is an architecture-first, mobile-first exam learning system tailored specifically for low-bandwidth, low-device-tier environments in India (tier-2, tier-3, and rural areas). Built to emulate and beat structural paradigms established by platforms like NTA Abhyas and Adda247, the product guarantees flawless functional continuity without active networks while supplying personalization through a dual-layer AI network.

The objective is clear: allow a student to walk into an area of zero network coverage, spend weeks studying, executing complex mock examinations, accessing granular cross-lingual deep explanations, analyzing systemic performance deficits, and updating their localized cognitive profile without hitting a single network timeout.

> **Core Operational Non-Negotiables:**
>
> - **Zero-Failure Offline Paradigm:** All local sqlite writes, transaction queues, package extraction structures, and runtime test trackers must maintain complete resilience against data loss or application lifecycle destruction by the Android OS.
> - **Mandatory Review & Explanations:** Every single test question must deliver contextually dense bilingual explanations (Hindi, English, Hinglish options) matching strict user profile parameters.
> - **No Pure Generative Inventions:** The AI layer is physically restricted from generating standalone raw technical solutions out of thin air. Everything relies heavily on a highly deterministic Retrieval-Augmented Generation (RAG) loop based on structured question banks.

2. Dual-Layer AI Architecture

To preserve extreme responsiveness on lower-end Android components while extracting structural synthesis from cloud models, the system divides processing payloads into a clear architectural split:

On-Device (Edge AI) Layer

- **Query & Intent Classification:** Runs local model runtimes via `Google AI Edge / LiteRT-TFLite` to classify local input streams instantly into functional intent buckets (e.g., `concept_doubt`, `formula_lookup`, `translation_request`).
- **Semantic Local Search:** Compact sentence embedding models mapped onto an on-device index interface (SQLite FTS5 + local vector similarity logic) to pull information directly from downloaded packs without network overhead.
- **On-Device Explanation Paraphrasing:** Highly optimized bilingual linguistic adapters to shorten or simplify existing local explanations for users requiring direct text synthesis.
- **OCR Pipeline:** On-device extraction utilizing `ML Kit OCR Core` to scrape text strings from snapshot camera captures of localized practice datasets.

Cloud AI Layer

- **Daily Schedule Synthesis:** Heavier programmatic calculations handling macro variables (weak performance dimensions, macro countdown clocks, consistency vectors) to output clean relational schema representations for local insertion.
- **Content Pipeline Synthesis:** Machine translation pipelines, validation operations, and multi-distractor generation processes executed inside secure sandboxes before publishing updates to the content storage structures.

3. Structural Module Specifications

Module A: User Onboarding & Dynamic Profiling

Provides zero-friction account generation coupled with direct local configuration metrics initialization.

- **Features:** Phone number verification via background-managed SMS OTP fallback structures; Exam selector matrices (SSC, Banking); Multi-tiered language selection (English, Hindi, mixed Hinglish strings); Daily study budget capture (30m, 1h, 2h, 3h+); Static self-reporting tracking matrix for weak domains.
- **Execution Profile:** The onboarding wizard compiles an immediate local profile transaction. This configuration object is persisted in SQLite instantly before any network synchronization thread fires, bypassing remote latency errors.

Module B: Relational Bilingual Question Bank

Forms the backbone repository pattern. Every query item must support complex relational mapping.

Field Attribute	Data Type / Mapping	Functional Definition & Intent
`id`	UUID / Primary Key	Global immutable question identification tag across cloud and client dbs.
`content_bilingual`	JSONB / Structured Object	Strictly localized structures separating explicit `"en"` and `"hi"` text nodes.
`options_bilingual`	JSONB / Array Object	Maintains keys for options A through E with explicit dual-language variants.
`correct_option`	VARCHAR(2)	The verified system choice key matching option parameters.
`explanations`	JSONB Block	Separated fields storing rich Markdown for English, Hindi, and localized Hinglish.
`distractor_notes`	JSONB / Optional	Granular contextual notes detailing exactly why distractor choices fail.
`metadata_tags`	Array / Relational Keys	Maps directly to Subject, Topic, Subtopic taxonomy links + PYQ indicators.

Module C: Diagnostic Test Engine

Establishes baseline capability mappings across target performance metrics using an adaptive framework.

- **Features:** Compressed 20-minute multi-disciplinary diagnostic packs containing standard mixed distribution arrays (Quant, Reasoning, English, General Knowledge).

- **Confidence Capture Metric:** Users can explicitly flag state nodes per attempt: `"sure"`, `"guessed"`, or `"not sure"`. This vector updates mathematical priority scoring weights inside the local metadata database.

Module D & E: Offline Test Execution & Advanced Review Engine

Replicates functional user patterns validated by industry standard applications, optimizing for zero network operations during runtime blocks.

- **Runtime Stability:** App backgrounds do not reset structural chronometers. Progress blocks and dynamic option capture matrices write directly to flash filesystems in real-time.
- **The Deep Review Interface:** Renders performance diagnostics cleanly immediately following submission. Compiles accurate local speed-accuracy vectors, pulls local taxonomy files, and exposes step-by-step trick workflows, structural formulas, and common trap indicators.

Module F: Spaced-Repetition Error Notebook

Captures client failure instances instantly. If an item returns an invalid evaluation vector against a correct answer key during an execution phase, it is targeted by a multi-tier local scheduler loop.

- **Scheduling Sequence:** Automates re-injection cycles along predefined interval offsets: $T + 1$ day, $T + 3$ days, $T + 7$ days, and $T + 14$ days.
- **Vocabulary & Formula Mechanics:** Converts specific metadata payloads into flashcard views readable inline without network queries.

4. Technical Database Architecture

The physical system uses a synchronized schema layout. The cloud database uses high-performance PostgreSQL fields, while the client uses a performance-tuned SQLite implementation accessible through low-overhead model layers.

Core Relational Schema Blueprint (PostgreSQL DDL)

```
-- 1. Base User Profile Entity
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone_number VARCHAR(15) UNIQUE NOT NULL,
    selected_exam VARCHAR(32) NOT NULL, -- 'SSC', 'BANKING'
    ui_language VARCHAR(10) NOT NULL DEFAULT 'hinglish',
    target_exam_date DATE NOT NULL,
    daily_capacity_minutes INTEGER NOT NULL DEFAULT 60,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Hierarchical Taxonomy System
CREATE TABLE taxonomy_nodes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    parent_id UUID REFERENCES taxonomy_nodes(id) ON DELETE CASCADE,
    node_level VARCHAR(16) NOT NULL, -- 'SUBJECT', 'TOPIC', 'SUBTOPIC'
    name_en VARCHAR(128) NOT NULL,
    name_hi VARCHAR(128) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Core Bilingual Question Entity
CREATE TABLE questions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    taxonomy_id UUID NOT NULL REFERENCES taxonomy_nodes(id),
    question_en TEXT NOT NULL,
    question_hi TEXT NOT NULL,
    options_en JSONB NOT NULL, -- {"A": "...", "B": "..."}
    options_hi JSONB NOT NULL, -- {"A": "...", "B": "..."}
    correct_option VARCHAR(2) NOT NULL,
    explanation_en TEXT NOT NULL,
    explanation_hi TEXT NOT NULL,
    explanation_hinglish TEXT,
    common_mistake_note TEXT,
    shortcut_formula_note TEXT,
    difficulty_level VARCHAR(12) NOT NULL, -- 'EASY', 'MEDIUM', 'HARD'
    estimated_solve_time_seconds INTEGER DEFAULT 60,
    version_integer INTEGER NOT NULL DEFAULT 1,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 4. Test Structure Manifest
CREATE TABLE tests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(256) NOT NULL,
    test_type VARCHAR(24) NOT NULL, -- 'DIAGNOSTIC', 'QUIZ', 'MOCK'
    duration_minutes INTEGER NOT NULL,
    is_packable BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. Test Relational Mapping Entity
CREATE TABLE test_question_mapping (
    test_id UUID REFERENCES tests(id) ON DELETE CASCADE,
    question_id UUID REFERENCES questions(id) ON DELETE CASCADE,
    sequence_order INTEGER NOT NULL,
    PRIMARY KEY (test_id, question_id)
);

-- 6. Unified Local-to-Cloud Sync Operations Ledger
CREATE TABLE synchronization_ledger (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    device_id VARCHAR(256) NOT NULL,
    event_type VARCHAR(32) NOT NULL, -- 'TEST_SUBMIT', 'BOOKMARK_ADD', 'PROFILE_UPDATE'
    payload_json JSONB NOT NULL,
    processed_status BOOLEAN DEFAULT FALSE,
    client_timestamp TIMESTAMP WITH TIME ZONE NOT NULL,
    server_timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

5. Implementation & Sprint Sequence Matrix

The production pipeline is prioritized to deliver functional foundational components first. Development is broken down into four distinct parallel tracks across five core execution milestones.

Milestone	Dev 1: Mobile (Flutter)	Dev 2: Backend & Sync	Dev 3: AI/ML & Edge	Dev 4: Admin & Web CMS
Sprint 01	Initialize local SQLite database structures, model mirrors, and foundational routing workflows.	Generate baseline PostgreSQL configurations and build schema architecture.	Implement text conversion workflows and structural input verification tests.	Build initial question generation layout tools and core text storage blocks.
Sprint 02	Build the core offline examination engine UI, local timer services, and answer capture workflows.	Create binary packaging compilation workers and endpoint sync targets.	Deploy `LiteRT-TFLite` model inference steps inside the mobile execution framework.	Deploy CSV/JSON secure transfer modules for the core question repositories.
Sprint 03	Develop the post-test analytics screen, detailed metrics views, and localized search layouts.	Build asynchronous batch queue microservices using `Redis` backend workers.	Set up SQLite FTS5 local text indexing engines to enable offline search execution.	Deploy batch assignment panels for partner coaching institutional views.
Sprint 04	Build the local data sync status broker and background queue monitoring layers.	Finalize optimization layers for high-volume concurrent write processes.	Connect localized `ML Kit` text extraction utilities into the input flow.	Integrate analytical tracking pipelines and generate exportable data reporting frames.

6. Release Checklists & Quality Gates

Pre-Launch Validation Milestones

- **Cold-Start Simulation Profile:** Verify the application successfully completes local database compilation and launches within 2.8 seconds on devices with limited RAM hardware (e.g., 3GB specs running Android 10).
- **Atomic Network Separation Quality Test:** Disconnect any wireless access interfaces physically during active evaluation operations. Confirm absolutely zero UI render blocks or exception alerts occur across 120 minutes of continuous runtime.

- **Data Integrity Validation Loop:** Force sudden thread context interruptions and app terminations during write processes. Confirm the atomic transactional SQLite properties prevent data contamination or profile structure drops.

Data Synchronizer State Invariant Rules

All dynamic sync components must adhere to strict monotonic ordering guidelines. A local device event sequence tracking payload $E = [e_1, e_2, \dots e_n]$ ordered by transaction timestamps t_{client} must maintain its exact relative structure when evaluated by backend verification layers. Conflict resolution defaults strictly to an evaluation logic where specific local update markers take precedence over stale, out-of-date remote fields.